理论 7：有限状态机的抽象和转换

问题是什么

读取一个很长的二进制字符串时，怎样判断它表示的数能否被 3 整除，而不把整个数保存下来？关键是只保留影响未来判断的信息：当前余数。这个例子引出有限状态机，也引出另一个问题：如果状态机和正则表达式描述的是同一类字符串，能否在两种表示之间机械地转换？

本项目分 Basic 与 Advance 两个阶段。Basic 只需完成第 1 周内容：从二进制模三判定抽象出状态、转移和接受条件；Advance 完成其余内容：DFA、NFA、正则语言、状态消除法、转换验证与抽象的局限。自动生成正则表达式的完整程序作为 Bonus，不要求实现工业级正则引擎。

阶段	工作量安排	完成后的能力
Basic	一周内约 10-20 小时	能设计模三状态机，用不变式解释正确性，并完成流式判定程序
Advance	完成 Basic 后，再用六周推进（总计七周）；建议难度系数 0.8，以 project1 的 1.0 为基准	能手工完成状态消除，解释正则表达式与有限自动机的等价关系

Basic：把历史压缩成状态

Basic 的核心概念包含：状态、字母表、状态转移、初始状态、接受状态与不变式。完成 Basic 时，须按 **Q1-Q3** 提交简短回答。

编号	需要回答的问题	回答应包含的内容
Q1	为什么判断模三余数只需要三个状态?	当前前缀的余数只有 0、1、2 三种；未来余数只依赖当前余数和下一位，不依赖完整历史
Q2	读取新位 b 后，状态为什么更新为 (2r+b) mod 3?	二进制追加一位等于原值乘 2 再加该位；模运算允许用余数代替原值计算
Q3	怎样说明程序对任意长度的合法输入都正确?	写出“状态等于已读前缀的模三余数”的不变式；验证初始情况、一步保持与最终接受条件

关键区分：

- 状态保存的是足以决定未来行为的信息，不一定是所有历史数据。
- DFA 每个状态对每个输入符号有一个确定后继；NFA 可有多个后继或空串转移，但不因此获得更强的语言识别能力。
- 理论正则表达式只讨论并、连接与 Kleene 星等正则运算；带反向引用的工程语法不在这一等价结论内。
- 有限测试可发现转换错误，不能证明两个表示对所有字符串等价。

模三状态表与输入约定

当前状态	读到 **0**	读到 **1**	含义
q0	q0	q1	已读前缀余数为 0
q1	q2	q0	已读前缀余数为 1
q2	q1	q2	已读前缀余数为 2

初始状态与唯一接受状态均为 q0。为方便讨论语言，约定允许前导零，并把空串的数值视为 0，因此空串也被接受。若设计用户输入接口，可另外拒绝空输入，但需明确这改变了所识别的语言。非 0/1 字符应报告非法输入。

Basic 手工跟踪 `110`、`101`、`000` 和空串，编写逐位更新程序，不先把整个输入转成大整数。提交 Q1-Q3、状态表、运行轨迹与代码。

Advance：从自动机到正则表达式

Advance 在 Basic 基础上讨论有限记忆的统一模型，以及如何在不同表示之间保持语言不变。

建议完成以下任务：

1. 用五元组 $(Q, \Sigma, \delta, q_0, F)$ 表示 DFA，并将模三问题逐项对应到模型。
2. 另设计一个识别“以 01 结尾”的二进制字符串的 DFA，比较它保留的历史与模三状态的差别。
3. 理解正则表达式中的并、连接、星号，以及空语言与只含空串的语言之间的区别。
4. 把一个少量状态的 DFA 改成广义非确定有限自动机（GNFA），添加新的唯一开始与接受状态。
5. 手工完成一个状态消除案例，逐步记录边标签与路径含义；至少有一次消除涉及自环或并行路径。
6. 对转换前后的表示枚举短字符串进行交叉验证，出现不一致时输出最短反例。
7. 完成研究报告和 presentation，说明表示转换、状态抽象与自动化之间的关系。

表示与转换

表示或操作	核心作用	需要掌握的边界
DFA	每读入一个字符，更新唯一当前状态	显式转移表下，长度为 n 的串可按 n 步扫描
NFA 与 ε-NFA	用可能状态集合表达分支与空串转移	模拟时需维护状态集合与空串闭包
NFA 转 DFA	以原状态的子集作为新状态	最坏情况下状态数量会指数增长
正则表达式转自动机	把并、连接和重复转成图结构	需明确采用的正则语法
状态消除	用正则边标签概括经过已删状态的路径	不同消除顺序可能产生大小不同的表达式
DFA 最小化	合并无法被任何后缀区分的状态	不是仅凭状态名字或相似边就合并

状态消除的更新规则

设 R_{ij} 是当前从状态 i 到 j 的边标签。消除状态 k 时，对仍保留的 i、j 更新：

$$R_{ij} \leftarrow R_{ij} \cup R_{ik}(R_{kk})^* R_{kj}.$$

右侧分别表示原有路径，以及进入 k、在 k 重复若干次、再离开 k 的路径。空缺边用 $\varnothing$ 表示，空串用 ε 表示；特别注意 $\varnothing^* = \varepsilon$。同一步更新应使用消除前的标签，避免原地修改混入新结果。

例如从开始点经 `a` 到中间点，中间点有 `b` 自环，再经 `c` 到接受点，结果为 `ab*c`。若还存在一条直接标记 `d` 的路径，就需要把两条路径取并。解释这一规则为什么保持开始点到接受点的语言。

实验与提交要求

必做内容包括两个 DFA、模三判定程序、一个完整手工消除过程，以及转换前后的短串交叉测试。建议枚举长度 0-10 的全部二进制串，包含空串、前导零、接受与拒绝样例；使用正则库时调用完整匹配，而不是子串搜索。

报告必须说明测试覆盖范围。若要进一步证明两个 DFA 等价，可构造乘积自动机，检查能否到达“恰有一个分量接受”的状态；没有此类可达状态才构成完整的等价性判定。提交代码、状态图或状态表、转换记录、测试结果，以及研究报告和 presentation。

Bonus

方向	重点理解的问题
自动状态消除	如何表示空语言、空串与表达式语法树？怎样避免错误的括号省略？
推广模数	对固定模数 m、进制 b，为什么转移为 (br+a) mod m？状态数会怎样变化？
自动机的边界	为什么一般的括号配对需要无界计数或栈？有限状态机遗漏了什么能力？

Advance：在 Basic 基础上追加六周

时间	主要任务	**Checkpoint**
第 1 周	完成 Basic 模三状态机	提交 Q1-Q3、状态表、轨迹与程序
第 2 周	学习一般 DFA，完成第二个例子	每个状态有清楚的语义与完整转移
第 3 周	学习正则语言与状态消除	完成带自环或并行路径的手工转换
第 4-6 周	完成交叉验证，比较消除顺序	测试包括空串，并能输出反例
第 7 周	整理研究报告和 presentation	能解释抽象、转换与等价性的区别

附录：问题的来龙去脉与发展过程

从一个数论小题到有限记忆。模三判定似乎需要先读完整个数，但未来判断只依赖余数。这个压缩过程展示了抽象的核心：丢弃无关细节，同时保留所有影响后续行为的信息。

从转移图到统一模型。有限状态机可以描述词法识别、简单协议与控制流程。状态既可以来自物理过程，也可以由程序设计者提炼，关键是每个状态是否具有明确、可验证的含义。

从两种表示到等价性。正则表达式描述允许出现的串，自动机描述读取这些串时发生的状态变化。Kleene 的相关工作建立了两者之间的联系，使“模式描述”与“可执行识别过程”能够相互转换。

从手算到自动化与验证。状态消除把经过中间点的所有路径压缩为表达式；子集构造把不确定选择转成确定的集合状态。这些方法说明，表达能力相同的表示，其大小和运行成本仍可能差异很大。参见 Stanford 的正则表达式课程讲义。

参考材料

- 本地材料：v0 有限状态机项目。
- MIT 18.404J Theory of Computation，选读有限自动机与正则语言部分。